TECHNISCHE UNIVERSITÄT BERLIN

Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario

Master's Thesis

Author
Piotr Witkowski
414360
witkowski@campus.tu-berlin.de

Advisor
Trever Schirmer

Examiners
Prof. Dr.-Ing. David Bermbach
Prof. Dr. habil. Odej Kao

Technische Universität Berlin, 2025
Fakultät Elektrotechnik und Informatik
Fachgebiet Scalable Software Systems

Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario

Master's Thesis

Submitted by:
Piotr Witkowski
414360
witkowski@campus.tu-berlin.de

Technische Universität Berlin
Fakultät Elektrotechnik und Informatik
Fachgebiet Scalable Software Systems

2025

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generische KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z. B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 8. März 2017* habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

(Unterschrift) Piotr Witkowski, Berlin, 22. Februar 2025

*`https://www.tu.berlin/arbeiten/wichtige-dokumente/richtlinien-leitlinien`

Abstract

Kurzfassung

Contents

1 Introduction

The digital revolution has fundamentally transformed the way we create, consume, and interact with visual information. Images are no longer only physical items around us, they are inherent components of our online experience, shaping perceptions and influencing decisions. As the Internet continues to evolve, so too grows the demand for rich and engaging visual content. This demand, however, comes at a cost. Images, while visually appealing, are significantly impacting website performance, not only by generating the highest number of requests, but also by consuming the most bytes during page load of an average site. Existing studies have shown a strong correlation between page load times and user engagement, conversion rates, and even search engine rankings. Minimizing image size is therefore fundamental for efficient bandwidth use and plays a crucial role in optimizing website performance.

Image optimization practices for the web have evolved over time, following developments of the state-of-the-art image formats and codecs. These solutions perform operations such as image resizing and format conversion, and may allow for storing optimized versions of original files for future use. While the optimization itself is a relatively specialized task, its deployment models follow familiar and established patterns in the general software industry: we can find them in software-as-a-service offerings, integrated into web hosting and content delivery platforms, but also as open-source solutions for self-hosting.

This last option, leveraging open-source image processing libraries, presents a compelling opportunity for our research. Unlike proprietary solutions, open-source projects offer the flexibility to be deployed across a variety of environments, granting builders granular control over the optimization process. This allows not only to measure influence of different libraries and optimization parameters, but also – and crucially for this thesis – enables realistic benchmarking of the underlying platforms on which these operations are performed. By employing these workloads, our research aims to accurately measure cloud service performance under the realistic conditions of real-world image optimization scenarios.

Given the event-driven nature of the optimization workload in response to requests for images, serverless cloud compute emerges as a well-suited deployment option for these systems. However, the varying demands for image processing, including different image sizes and diverse range of image formats present on the web, raise questions about the suitability and performance consistency of these platforms for the use-case. Furthermore, the emergence of function as a service (FaaS) platforms introduces a new dimension to cloud-based, self-hosted image optimization. FaaS offers a compelling execution environment for image processing tasks thanks to its on-demand scalability and event-driven nature. However, the performance variability inherent in FaaS platforms, attributed to diverse factors, may impact efficiency of the process and applicability of the environment for the use case.

1.1 Objectives of the Thesis

This thesis aims to investigate whether FaaS performance variability has measurable and meaningful influence for image optimization workloads. We will define set of representative metrics related to image optimization, reflecting realistic web usage. We will design and implement the optimization workflows for selected serverless environments. We will design and implement scheduling mechanism allowing us to simulate requests for optimized images. We will collect performance metrics across all parameters, apply statistical methods

for result comparisons, and visualize the results. We will investigate if more intensive optimization can be cost-efficient, considering optimization result caching, and content reuse for multiple viewers. We will provide recommendations that are within the scope of the benchmark, and answer the primary research question in the context of the scenario.

1.2 Structure

2 Background

This chapter provides readers with foundations for understanding the challenges and opportunities of serverless image processing. We begin by revisiting fundamental principles of imaging and photography. Subsequently, we describe core digital imaging concepts, emphasizing aspects relevant to web performance. We then examine the characteristics of serverless cloud computing, focusing on factors contributing to performance variability. Finally, we explore image optimization for the web, reviewing established techniques and the landscape of cloud-based solutions.

2.1 Imaging and Photography

Imaging is a broad term describing any process that helps us in visualizing and understanding physical objects, abstract concepts, or even sets of data. While this may be a purely philosophical activity, for the purpose of this thesis, we will define imaging as creating a visual representation - an image - of physical objects and scenes. Humans are able to perceive and interpret these images thanks to our visual system, that has evolved over millions of years [LN12]. Developed from the simple light-sensitive cells of early organisms to the sophisticated eyes of modern humans, this system is providing us with the ability to see color, depth, or movement [LH88].

From the earliest cave paintings [Wac93] to modern photography and film, humans have been exploring ways to capture and recreate the visual world. Over centuries, scholars and artists showed great interest [HF10] in the process of creating visual representations to understand our own mind and senses, as well as to be able to depict the world around us more accurately. The natural phenomenon of light passing through a small aperture, called camera obscura, helped us understand fundamental principles of optics and image formation, and according to some [Hoc01], greatly contributed to realism of European art during Middle Ages. We've been refining tools and techniques to enhance our vision, and improve everyday life. Devices like telescopes and microscopes allowed us to observe objects at different scales, unveiling worlds that would otherwise remain invisible to the naked eye. Eyeglasses and contact lenses correct refractive errors, demonstrating that our visual experiences can not only be externally altered, but also subjectively improved, for millions of people.

Photography emerged in the 19th century as a groundbreaking invention, offering a novel approach to image creation. Unlike paintings or drawings, which rely on an artist's interpretation, photography directly captures and permanently records the physical world using light-sensitive materials. Early photographic methods required long exposure times, often making it possible to photograph only still objects and posed portraits [Dav99]. The daguerreotype, which used a silver-coated copper plate, was one of the first widely adopted photographic processes. Later, more versatile film technologies significantly reduced exposure times and made photography more accessible to the wider public. This ability to easily and accurately record the physical world had significant consequences. Newspapers, now able to include actual photographs, found a powerful new way to share news and social commentary through photojournalism [GM20]. On a personal level, photography allowed people to preserve their own histories through portraits and the documentation of special occasions.

Since the beginning of the 21st century, digital photography has become the dominant form [Zdn03] of capturing and sharing images. Initially limited to standalone devices,

digital cameras are now integrated into smartphones and personal computers. Electronic sensors capture and convert light into digital data, facilitating easy manipulation and image sharing, including over the Internet. This digital nature has led to the popularity of standardized formats, ensuring compatibility across diverse platforms.

However, the ease of image creation and sharing, coupled with the vast amount of image data generated daily, may present unexpected challenges for storage and transmission. In 2024, an estimated 6 billion photos are taken daily [Bro24], with 14 billion images shared on social media platforms [Ric17]. Especially in the context of thesis, the resulting volume of image data necessitates efficient delivery. Fortunately, optimization methods exist, relying on core concepts of image representation, encoding, and compression, to ensure efficient delivery across networks. We will explore these in the subsequent sections.

2.1.1 Digital imaging

Digital imaging, like traditional photography, relies on the fundamental principle of capturing light to create a visual representation. Most image acquisition methods, whether they involve cell phone cameras, DSLR (digital single-lens reflex) and mirrorless cameras, or even scanners, are variations of the classical optical camera [WD05]. Concepts such as the relationship between aperture and shutter speed to achieve proper exposure are universal, regardless of whether the capture method is analog or digital.

However, digital imaging distinguishes itself by representing images as discrete, two-dimensional arrays of numerical data. This involves transforming light into a grid of distinct pixels. Each pixel may contain one or more channels — components that record different aspects of light, often corresponding to how our eyes perceive color. If only one channel is present, the image is effectively grayscale, as was the case with the first digital camera invented in 1975 by Steven Sasson [ORe18].

To create a digital image, the camera must first capture the light that's focused onto its sensor. The position of the camera and its lens determine what portion of the scene is captured by each pixel on the sensor — this is known as *spatial sampling*. The camera also controls how long it gathers light at each pixel, called *temporal sampling*, and is determined by the exposure time, or using photographic terminology, *shutter speed*. Thanks to the photovoltaic effect, the amount of the light captured can be converted into digital value in a process called *quantization* [BB22]. The number of distinct values that can be represented for each pixel depends on the bit depth of the image.

Digital cameras primarily use two image sensor technologies: CCD (Charge-Coupled Device) [The95] and CMOS (Complementary Metal-Oxide-Semiconductor) [Fos97]. Early digital cameras utilized CCD sensors, which offer high sensitivity and low noise due to their charge transfer method, where all charge is shifted across the chip to a single output node. However, this sequential readout makes them slower for continuous shooting or video capture. CCDs also tend to consume more power and are more prone to blooming, where overexposed pixels overflow. In contrast, CMOS sensors have seen rapid development since the 1990s. While initially lagging in sensitivity and noise performance comparing to CCDs [Mag03], CMOS sensors have greatly improved through advancements like on-chip noise reduction. Their lower cost, lower power consumption, and faster readout speeds have led to CMOS sensors becoming more popular than CCDs and being found in a wide range of devices, from smartphones and webcams to DSLRs, mirrorless cameras, and even scientific imaging equipment [HL07].

To accurately capture color, most digital cameras use a *color filter array* (CFA) placed over the sensor. The most common type is the Bayer filter, named after its inventor, Bryce Bayer, working at the time at Eastman Kodak. This filter arranges red, green, and blue filters in a specific pattern over the pixels. Because the human eye is more sensitive to green light, or to be specific, because *green signal contributes more to the luminance signal than the red or blue signals* [Hun05], in a typical Bayer filter we can find twice as many green filters as red or blue. Cameras capture a full-color image by interpolating the missing color information for each pixel, in a process known as *demosaicing* [ASM20].

Following the Bayer filter example, it is common for color images to have three channels, such as red, green, and blue (RGB), each with its own bit depth. If each channel in an RGB image has an 8-bit resolution, it allows for 256 different values per channel, resulting in over 16 million possible color combinations for each pixel. The resolution of an image refers to the total number of pixels it contains and is often measured in megapixels (millions of pixels). Higher resolution images have more pixels, what allows for larger prints and more detailed displays. The image resolution, along with the bit depth, directly affects the size of the uncompressed image file. However, because images often contain areas of similar color, compression techniques, discussed in the next section, are frequently employed to reduce file sizes, sometimes even during the image capture process within the camera devices.

2.1.2 Data and image compression

Digital data, whether representing text, audio, or images, is fundamentally a collection of bits. Data compression techniques aim to represent the same information using fewer bits [Hof12], which is important for addressing physical constraints of digital systems, such as bandwidth, storage density, and power consumption. In 1948, C. Shannon introduced [Sha48] the concept of entropy as a measure of information content. Entropy defines the fundamental limit for lossless data compression algorithms, providing a theoretical target for minimizing the number of bits needed to represent information. To effectively leverage data compression techniques, it's important to differentiate between lossless and lossy compression, recognizing the trade-offs between preserving data integrity and achieving higher compression ratios.

Lossless compression methods enable reversible transformations of data into more compact forms, while preserving all original information. While this thesis primarily focuses on image processing, we would like to note that lossless techniques have much broader applications and can be applied to virtually any form of digital data, including text and binary files. This is crucial for efficient data transmission of any kind, where lossless compression reduces bandwidth requirements, but allows recipients to access the original information in its unaltered [Sal98] form.

Run-length encoding is a simple technique that exploits repetitions in data. For example, a sequence of the same 20 consecutive characters *a* can be efficiently stored as *(20, a)* instead of storing each individual character. This works especially well if the set of unique symbols in the data is small, and the data contains long *runs* of the same symbol.

Huffman coding [Huf52] takes a statistical approach by constructing a binary tree *(Huffman tree)* based on symbol probabilities. Leafs representing higher probability symbols are positioned closer to the root, with shorter code lengths. This results in a variable-length code, where the length of a symbol's bit sequence is inversely proportional to its frequency in the data. By assigning shorter codes to common symbols, Huffman coding optimizes code length, reducing the overall number of bits needed.

Arithmetic coding [Pas76; Ris78] takes yet another approach, encoding the entire message as a single fraction between the values of *0* and *1*. The exact value is determined by repeatedly subdividing the interval based on the probabilities of the symbols. Each symbol narrows the interval, and the final fraction precisely represents the entire sequence, often allowing it to be represented with fewer bits than the original message. However, this method requires the probability distribution of the symbols being encoded to be shared between the encoder and the decoder.

In addition to the methods described above that can help us understand the basics of compression, practical implementations of lossless compression often rely on dictionary-based techniques. The *Lempel-Ziv* algorithms, including LZ77 [ZL77], LZ78 [ZL78], and the *Lempel-Ziv-Welch* (LZW) [Wel84] algorithm, are important examples of this approach. LZ77 utilizes a sliding window to locate matches in preceding data, while LZ78 constructs a dictionary of previously encountered phrases. Combining LZ77 with Huffman coding, *Deflate* [Deu96] often achieves higher compression ratios than LZ77 alone.

Asymmetric Numeral Systems [Dud09; Dud14] (ANS) is a relatively new approach to entropy coding that combines the speed of Huffman coding with the compression rate of arithmetic coding. Unlike arithmetic coding, which uses two fractional numbers to represent an interval, ANS encodes data directly as a single natural number. This simpler representation, using just one integer, reduces computational complexity and allows for higher compression and decompression throughput. ANS is expected [Dud+15] to gain more popularity across various domains, including web delivery and image compression [HW22], as research [CK21] and implementation efforts continue.

Lossy compression methods, unlike lossless techniques, offer significantly higher compression ratios [Sal98] by selectively discarding information. This trade-off is often acceptable for media files like images and audio, where human perception of minor data alterations may be limited. At the same time, even smallest data alterations are not acceptable for other types of binary data, such as binary executable code, digital signatures or encryption keys.

Transform coding is a common technique used in lossy compression that serves two purposes. By compacting the original data samples into a series of transform coefficients with smaller and smaller variances, we are eventually able to remove the smallest coefficients with negligible reconstruction errors. Additionally, decorrelated coefficients can be quantized and entropy coded without losing compression performance. By applying a transform, such as the *Discrete Cosine Transform (DCT)* [ANR74], we can separate the information into components that vary in importance with respect to human perception. In lossy image compression, DCT selectively removes high-frequency components [WM23] which correspond to image details that are the least perceptible to human eyes.

Subsampling exploits the characteristics of human perception by reducing the spatial resolution of specific components within the data. In image compression, this often involves relative downsampling the chrominance components *(chroma)*, as the human eye is less sensitive to color detail compared to luminance *(luma)* detail. The reduction in chrominance resolution contributes to a smaller file size without significantly impacting the overall perceived image quality.

Evaluating the effectiveness of lossy compression involves measuring the perceived difference between original and compressed images. Selected evaluation metrics include Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Butteraugli [HZ10].

While metrics like PSNR and SSIM provide objective measurement of the differences between pictures, the ultimate measure of lossy compression effectiveness will eventually always depend on the subjective perception of its observers.

Another factor to consider with lossy compression is generation loss, which refers to the accumulated degradation of quality that occurs when a compressed file is repeatedly edited and recompressed. Each recompression cycle may introduce additional information loss, potentially leading to noticeable artifacts and a significant reduction in overall subjective quality of the image. In the next section, we will explore different image formats that utilize the lossy and lossless compression techniques described above.

2.1.3 Raster and vector graphics

Image formats are standardized ways of organizing digital image data and metadata within files. They ensure compatibility and allow different software, devices, and operating systems to work with the same images. Two fundamental categories of image formats are *raster* and *vector*.

Raster images [Fol96] are fundamentally composed of a grid of individual pixels, each representing a specific color. This structure directly corresponds to the way digital cameras capture images and is well-suited for representing photographs and other visuals with continuous tones. The term raster originates from the scanning pattern *(raster scan)* of CRT monitors, where the electron beam illuminates the screen pixel by pixel, line by line. In addition to color data, some raster formats incorporate an alpha channel [PD84], which controls each pixel's transparency. Image quality in raster formats is fundamentally connected to the resolution, measured by the total number of pixels. When scaling a raster image beyond its original size, preserving a fixed number of pixels can lead to a visible pixelation - a situation when individual pixels become visible to a naked eye. The alternative approach of introducing new pixels — *interpolation* [Han13] — also has drawbacks: traditional *generating a raster image with a higher resolution than its source* using pixel interpolation methods may cause a visible loss of sharpness [Ouw06], also known as *edge blurring*.

Vector graphics offer a fundamentally different approach, representing visuals using geometrical primitives like lines, curves, circles, ellipses, or polylines and polygons [Mor99]. These primitives are defined by their mathematical properties, not as individual pixels. For instance, a circle may be defined by its center point coordinates and radius, while a curve might be represented using its end point and control points. This allows vector graphics to be inherently scalable, as the same geometrical definitions are used to render the graphic at any size.

While the following sections of our thesis will focus on raster image optimizations, a brief comparison of image optimization techniques for both vector and raster images is beneficial for the comprehensive understanding of the topic. Pixel-based methods, such as reducing image resolution, the bit depth, or applying lossy compression can not be directly translated to their vector counterparts, as vector graphics do not rely on a pixel grid [Lib18]. Instead, byte size of vector graphics may be optimized through methods unique to their geometrical nature. While lossless compression methods still apply, further optimizations are typically achieved by reducing the precision of values defining geometrical primitives, simplifying complex shapes by using fewer defining points, and, in formats supporting reusable styles, minimizing the number of distinct styles used [Mar15].

2.1.4 Raster image formats

Building upon our understanding of various compression techniques and raster graphics, we will now explore specific raster image formats. While based on common data representation principles, specific formats use different strategies for storing and compressing pixel data.

The existence of multiple formats is driven by technological advancements in the field of image processing, evolving consumer needs, as well as industry competition. While new algorithms and standards allow for smaller file sizes and additional features, widespread adoption can only happen if the benefits of new image formats outweight compatibility issues and justifies additional investment (monetary and engineering) needed to enable and integrate the new technology into existing systems and workflows.

2.2 Cloud Computing

Various organizations and standardization bodies have formulated alternative definitions of cloud computing based on their specific priorities and perspectives. ISO/IEC 22123-1 [Sta23] defines cloud computing as a "paradigm for enabling network access to a scalable and elastic pool of shareable physical or virtual resources with self-service provisioning and administration on-demand". This definition, with the emphasis on essential characteristics like on-demand availability and self-service, shares similarities with the one provided by the National Institute of Standards and Technology in *The NIST Definition of Cloud Computing* [MG11]. However, NIST also provides a more focused categorization of cloud services, identifying three service models (Infrastructure as a Service - *IaaS*, Platform as a Service - *PaaS*, and Software as a Service - *SaaS*) and four deployment models (private, community, public, and hybrid clouds) instead of 7 service categories and 8 deployment models in the ISO standard. In contrast to the frameworks offered by ISO/IEC and NIST, commercial organizations like Amazon Web Services (AWS) tend to use more concise and business-oriented definitions, such as "on-demand delivery of IT resources over the Internet with pay-as-you-go pricing" [AWS24]. Despite these variations, all definitions focus on the core value proposition of cloud computing: providing flexible, accessible, and easy-to-manage computing services.

In a manner similar to services such as water and electricity supply, cloud computing can be seen as a utility and an on-demand enabler for innovation and agility without an upfront investment [Gro09] in one's own IT infrastructure. With access to public cloud infrastructures, organizations do not have to build and maintain their own data centers, much like how most individuals and businesses no longer need to build their own power plants. Instead, they can focus on their unique strengths and leverage the scale and efficiency of shared resources, essentially offloading the *undifferentiated heavy-lifting* [Kin23]. Furthermore, many cloud providers offer free tier access to a range of their services, allowing IT practitioners and business decision makers to experiment and gain familiarity with cloud environments before committing to substantial financial investments.

Cloud service models described earlier form a spectrum. Starting from the least *managed* types of services in the *IaaS* model, bare-metal servers and virtual machines offer maximum control, but require the highest degree of direct infrastructure management. In the *PaaS* model, developers and software vendors[1] trade some of this control for simplified operations [WTJ14]. It is still required to provide source code for applications, but the platform provider typically manages the operating system and offers additional capabilities with help of the platform-specific APIs and SDKs. In the *SaaS* model, a pre-built cloud service is solving a specific business problem for its users[2], with the least amount of flexibility comparing to previous models. SaaS services work well for repeatable workloads, such as project management software (Asana, Trello), communication and collaboration tools (Slack, Google Workspace), or e-commerce (Shopify) that can be applied with minimal modifications for a wide range of customers [Kav14].

No two cloud platforms are the same. Leveraging differentiating, proprietary features can accelerate development of cloud-based services, but at the same requires careful consideration before adoption. These features, often exclusive to specific providers, can lead to a situation where users inadvertently depend on a particular technology, known as *vendor lock-in* or *feature lock-in* [Pet11]. This dependency can pose a challenge when migrating

[1] By developers and software vendors, we mean personas setting up and maintaining cloud deployments.
[2] By users, we mean end-users of the services and applications deployed on the cloud.

to a different cloud vendor or pursuing a multi-cloud deployment strategy. Moreover, the perceived value and associated risks of such dependencies can change over time, influenced by evolving business needs and technological landscape.

Shared Responsibility Model is a framework adopted by independently by multiple public cloud providers. In its basic form, it refers to the security *of the cloud* and the security *in the cloud*. In this model, both cloud provider and cloud user share responsibility of different security aspects of the cloud-based workloads.

- Cloud provider is responsible for the physical security of the data center facilities, providing uninterrupted power and cooling supply, and maintaining integrity of the infrastructure. Cloud provider is also responsible for providing customers with building blocks to implement architectural best practices, such as resiliency to natural disasters with multiple regions and availability zones.
- Developers and software vendors are responsible, for example, for preparing disaster recovery strategy including platform support, following the principle of last privilege, timely updates of their application code, or applying application-layer encryption.

In addition to the above, we would like to note that the exact split in the shared responsibility model depends greatly on the used service delivery model. While things like power or cooling for IT resources are hidden from end users independent of the used model, many other responsibilities, similar to the lock-in discussion above, depend on whether users opt into abstractions provided by the cloud provider. Typically, the more *managed* a service or model is, the more responsibility is taken by the cloud service provider. Providers abstracting typical IT use cases, such as OS supervision, load balancing, and availability of the managed services according to the SLAs of individual services. This leads to less freedom in configurability, but instead, users can shift their development and operational effort towards developing high-value, differentiating capabilities in their domain of expertise. For example, when comparing bare-metal instances with virtual machines, users will be required to set up and maintain a hypervisor only in the first situation. On top of security, similar shared responsibility models can be applied to domains like compliance or even sustainability.

The following sections and the rest of our thesis will focus on serverless computing, a paradigm where, contrary to its name, servers still exist. Serverless operations are abstracted even further from the developers, therefore, as before, it is important to make an informed decision whether to opt into platforms' vendor-specific features. Existing literature, such as [Jon+19], explores the potential pitfalls and fallacies associated with serverless computing, providing a valuable context for understanding the broader implications of these decisions. Finally, adoption of cloud computing, like any technology, while potentially beneficial, must be evaluated with the user in mind. As Tim O'Reilly put it, "lock-in comes because others depend on the benefit from your services, not because you are completely in control" [ORe03] - it is the users who eventually benefit from, or are constrained by, all architectural choices.

2.2.1 Serverless computing

Serverless is an approach where cloud providers aim to simplify cloud operations even further. While sharing many similarities with the PaaS model, PaaS services typically operate on the application or microservice level. Serverless approach is instead concerned about individual capabilities or business functions within an application. In this model,

software vendors have very little control over actual hardware, for example size and type of the(virtual) machine, on which their business logic is executed. Capacity provisioning, capacity planning, architecting for redundancy and high availability, as well as detailed monitoring features are built into serverless platforms, and managed by the platform provider.

Two most popular service deployment models applicable to the serverless approach are:

- **Backend as a Service (BaaS)** model provides developers and software vendors with options to adopt cloud-based services directly by clients, such as websites and mobile applications. Examples of BaaS frameworks include Google's Firebase or AWS Amplify[3], and typical use cases include authentication, persistent storage coupled with user's identity, or notifications management. Implementation-wise, BaaS is similar to integrating with any other SDK, with the major difference of focusing on rather *core* capabilities of an application transparently for the users - often without making them aware about its reliance on a particular BaaS product, as if the application custom-made backend was used.
- **Function as a Service (FaaS)** enables developers and software vendors to run code without the need to manage the underlying infrastructure or runtime. Examples of FaaS services include AWS Lambda and Google Cloud Functions. Functions can be synchronous or asynchronous. Responding to an HTTP request is an example of the former, and event-based invocation is an example of the latter. Asynchronous functions are typically executed for their side effects, such as storing the result of a computation in an external data store. While functions can persist state between invocations, service providers may terminate execution environments at any time they are not actively handling an invocation. Therefore, functions should be designed to operate independently of any existing state within the execution environment. Sometimes, simply providing the function code is sufficient, with the FaaS platform managing dependencies and the execution environment. If the provider supports custom container images as the function source, users are not limited to any specific programming language or runtime when authoring functions, but they will need to build, deploy, and maintain these container images themselves.

Thanks to granular monitoring mentioned earlier, serverless services offer fine-grained, usage-based pricing models. These operate on dimensions such as individual requests, CPU time with high resolution (as low as 1 millisecond, which is not the case in IaaS or PaaS models), amount of specific API operations, or metrics such as MAU (monthly active users). On-demand capacity of serverless backends scales automatically, and in real-time, in response to traffic. However, all this flexibility, especially in the FaaS model, comes at a cost. *Limited lifetime* [Hel+18] of serverless functions prescribes that their internal state may not persist between invocations. True *on-demand scaling to zero* only works, if we accept the trade-off of potential cold starts. With provisioned capacity we can avoid cold starts, but scaling to zero will not be possible anymore. Similarly, a choice must be made to the question of using a built-in runtime versus a custom container image. All of such architectural choices have to balance the complexity of the deployment, cost, as well as performance [DKL22] of the resulting service.

[3]Similar to Google's Firebase, AWS Amplify lets users host static pages and predefined application backends. In contrast to Firebase, Amplify offers purely client-side wrappers for other managed AWS services, such as Amazon Cognito, Amazon API Gateway, or Amazon S3, to simplify client-side integrations with these services.

In FaaS deployments, minification and tree-shaking can be used to reduce the size of the deployment package, and therefore the cold start time [PS17]. Native libraries can be used within serverless functions, even without a custom container, but in such cases, it's crucial to ensure that the CPU architecture and operating system runtime of the serverless platform is compatible with the library code. By leveraging such libraries, developers can avoid typical performance penalty associated with scripting languages [Ous98], while enjoying convenience of high-level language to handle an event. To sum up, serverless computing offers a compelling approach to cloud operations, shifting the focus from infrastructure management to individual functions or capabilities. While BaaS simplifies client-side integrations with core application features, FaaS allows code execution without runtime management.

In the following sections, when speaking about serverless, we will focus exclusively on the Function as a Service model, as it is more relevant to the self-hosted image optimization use case we are exploring in this thesis.

2.2.2 Cloud service benchmarking

Cloud service benchmarking is the process of systematically evaluating and comparing different qualities of cloud-based services. Quality can be defined as a measure of any non-functional property of a service. In other words, qualities tells us not *if*, but *how good* a system fulfills its purpose. Qualities may describe radically different aspects of a system, for example, measure the error rate of a given operation or degree of compliance with a particular standard or regulation. While importance of a quality depends on a specific system and application, qualities in general express the difference from an ideal, sometimes purely abstract reference point. Continuing with the previous examples, an ideal system or service of the highest quality would yield an error rate of 0 (no errors, or undesired results), and demonstrate complete alignment with, or no exception from, a certain specification.

While *monitoring* is a continuous, low-impact observation of the production environment, used to detect undesirable system states [BWT17], *benchmarking* answers specific questions about the system under test (SUT). It is often a result of an offline analysis, and based on the recording of arbitrary metrics. In benchmarking, a separate, non-production environment is used, as certain operations, for example stress-testing or fault injection, may be too risky and disruptive in a production setting or result in undesired consequences, such as a service outage or revenue loss.

Benchmarking allows for the evaluation of different configurations within the same environment, such as testing various memory allocations of a serverless application. It also enables comparing different systems while maintaining consistent configuration parameters, for example, measuring performance across multiple cloud providers or regions. This comparative nature helps identify optimal settings for specific use cases and understand the trade-offs between different implementation choices. By keeping some variables constant while varying others, benchmarking provides insights into how different factors influence system behavior.

Because cloud services are consumed over the Internet, designing benchmarks from the client perspective that closely reflect later use is essential to get meaningful results. In particular, even if additional information about the system is available (for example, source code of the application), literature [BWT17] suggests treating the underlying cloud platform and application deployed on it as a *black box*. The reliance on the service's external interface / API contract, rather than depending on the actual implementation, allows us to compare

different versions of the same service, and also align as closely as possible to the actual consumers' quality experience.

To facilitate benchmarking, a few components of a *benchmarking system* are needed. A *workload generator* is responsible for creating a realistic load on the system under test, as otherwise the system is idling. An *experiment controller* automates the execution of benchmark runs, managing configurations and collecting results. Finally, a *measurement database* stores the collected performance data, enabling analysis and comparison across different scenarios.

2.2.3 Performance variability of serverless workloads

Performance variability is a key consideration in Function as a Service (FaaS) environments due to the high level of abstraction and limited control over the underlying infrastructure. Existing literature has explored and identified several sources of this performance variability within serverless platforms, including [Llo+18] studying cold and warm service performance, [UAG21] researching influence of storage accesses and bursty function invocations, and [Sch+23] exploring variations in functions' execution duration, and the frequency of unexpected cold starts depending on temporal performance variability patterns. [Wan+18] investigates performance isolation efficiency of different serverless platforms.

One of the primary contributors to performance fluctuation is the phenomenon of *cold starts*. When a function is invoked for the first time (or after a period of inactivity), the cloud provider must allocate resources, initialize the runtime, and load the function code. Subsequent "warm" invocations can reuse this environment, leading to significantly lower latency. However, FaaS providers offer no guarantees about environment lifetimes, meaning cold starts can occur unpredictably. Mitigation techniques like provisioned concurrency exist but introduce trade-offs in cost and resource utilization. Research [DKL22] shows that the cold start latency depends on the deployment method (ZIP deployments being preferable for smaller Node.js, Python, and all Java functions, and container deployments for larger Node.js and Python functions), as well as the used programming language. Java functions, in particular, are known to exhibit longer cold start times due to the overhead of the Java Virtual Machine (JVM) [WJ22]. While the JVM's size and startup contribute to this latency, mitigation strategies such as Ahead-of-Time (AOT) compilation (e.g., with GraalVM) and memory snapshots (e.g., AWS Lambda SnapStart) can reduce this delay.

Beyond cold starts, variability in resource allocation [GF20] also impacts performance. Cloud providers dynamically allocate resources (CPU, memory, network) to each function invocation. Multi-tenancy and hardware differences can lead to inconsistent execution times even for identical workloads. Network latency, especially when functions interact with other services and APIs, adds another layer of unpredictability due to factors like congestion and variable geographical location (for example, in different regions, or availability zones). Benchmarking from a client perspective is crucial to capture realistic performance.

Concurrency and queuing effects further contribute [SBW19] to variability. Simultaneous invocations can lead to queuing particularly during peak loads, as the cloud provider's orchestration layer manages resources. These peak loads often exhibit periodic patterns, influenced by human activity and the scheduling of recurring tasks, which tend to cluster at common intervals (e.g., the start or end of hours, days, or weeks).

Due to the inherent abstraction of the FaaS model, ensuring consistent performance is a critical requirement for providers, as it is a prerequisite for predictable application behavior

and end-user experience. However, FaaS platform users can also influence workload performance, for example, by strategically scheduling workloads to avoid known periods of high usage mentioned above.

2.3 Image Optimization for the Web

Image optimization for the Web is

2.3.1 Impact of images on browsing experience

2.3.2 Bandwidth usage, visual quality, and browser support evolution

2.3.3 Optimizing latency of image delivery

2.3.4 Deployment methods for cloud-based image optimization

3 System Design

3.1 Image processing workload design

3.1.1 Serverless environment selection

3.1.2 Image manipulation library selection

3.1.3 Input image and output image selection

3.2 Benchmark design

3.2.1 Serverless platform and underlying hardware

3.2.2 Time of execution

3.2.3 Output formats and image widths

3.2.4 Available CPU / memory

3.2.5 Cold starts

3.3 Workload generation and benchmark execution

3.3.1 Serverless benchmark deployment

3.3.2 Scheduling workload executions

3.3.3 Results collection and data analysis

4 Results

4.1 Performance metrics

4.1.1 Cold start duration

4.1.2 Image processing duration

4.1.3 Memory utilization

4.2 Performance variability

4.2.1 General variability in performance metrics

4.2.2 Temporal variability

4.2.3 By image format

4.2.4 By platform configuration

5 Discussion

5.1 Performance variability analysis

5.2 User vs operator perspective

5.3 Cloud computing as a utility

6 Related Work

6.1 Serverless performance research

6.2 Image processing research

7 Conclusion

References

[ANR74] Nasir Ahmed, T. Raj Natarajan, and Kamisetty R. Rao. "Discrete Cosine Transform". In: *IEEE Transactions on Computers* C-23.1 (1974), pp. 90–93.

[ASM20] Luis A. Aranda, Alfonso Sánchez-Macián, and Juan A. Maestro. "A Methodology to Analyze the Fault Tolerance of Demosaicking Methods against Memory Single Event Functional Interrupts (SEFIs)". In: *Electronics* 9.10 (2020), p. 1619.

[AWS24] AWS. "What is Cloud Computing?" In: *AWS Whitepaper: Overview of Amazon Web Services* (2024).

[BB22] Wilhelm Burger and Mark J Burge. *Digital Image Processing: An Algorithmic Introduction*. Springer Nature, 2022.

[Bro24] Matic Broz. *Photo Statistics: How Many Photos are Taken Every Day?* 2024.

[BWT17] David Bermbach, Erik Wittern, and Stefan Tai. *Cloud Service Benchmarking*. Springer, 2017.

[CK21] Y. Collet and M. Kucherawy. *RFC 8878: Zstandard Compression and the 'application/zstd' Media Type*. 2021.

[Dav99] Alma Davenport. *The History of Photography: An Overview*. Unm Press, 1999.

[Deu96] Peter Deutsch. *RFC1951: DEFLATE Compressed Data Format Specification version 1.3*. 1996.

[DKL22] Jaime Dantas, Hamzeh Khazaei, and Marin Litoiu. "Application Deployment Strategies for Reducing the Cold Start Delay of AWS Lambda". In: *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. IEEE. 2022, pp. 1–10.

[Dud+15] Jarek Duda, Khalid Tahboub, Neeraj J. Gadgil, and Edward J. Delp. "The use of asymmetric numeral systems as an accurate replacement for Huffman coding". In: *2015 Picture Coding Symposium (PCS)*. IEEE. 2015, pp. 65–69.

[Dud09] Jarek Duda. *Asymmetric numeral systems*. 2009. arXiv: `0902.0271`.

[Dud14] Jarek Duda. *Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding*. 2014. arXiv: `1311.2540`.

[Fol96] James D. Foley. *Computer Graphics: Principles and Practice*. Vol. 12110. Addison-Wesley Professional, 1996.

[Fos97] Eric R. Fossum. "CMOS Image Sensors: Electronic Camera-On-A-Chip". In: *IEEE Transactions on Electron Devices* 44.10 (1997), pp. 1689–1698.

[GF20] Samuel Ginzburg and Michael J Freedman. "Serverless Isn't Server-Less: Measuring and Exploiting Resource Variability on Cloud FaaS Platforms". In: *Proceedings of the 2020 Sixth International Workshop on Serverless Computing*. 2020, pp. 43–48.

[GM20] Thierry Gervais and Gaëlle Morel. *The Making of Visual News: A History of Photography in the Press*. Routledge, 2020.

[Gro09] Robert L. Grossman. "The Case for Cloud Computing". In: *IT professional* 11.2 (2009), pp. 23–27.

[Han13] Dianyuan Han. "Comparison of Commonly Used Image Interpolation Methods". In: *Conference of the 2nd International Conference on Computer Science and Electronics Engineering (ICCSEE 2013)*. Atlantis Press. 2013, pp. 1556–1559.

[Hel+18] Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. "Serverless Comput-

ing: One Step Forward, Two Steps Back". In: *arXiv preprint arXiv:1812.03651* (2018).

[HF10] Hugh Honour and John Fleming. *The Visual Arts: A History*. Pearson Education, 2010.

[HL07] Gerald C. Holst and Terrence S. Lomheim. *CMOS/CCD Sensors*. JCD publishing, 2007.

[Hoc01] David Hockney. *Secret Knowledge: Rediscovering the Lost Techniques of the Old Masters*. Thames & Hudson London, 2001.

[Hof12] Roy Hoffman. *Data Compression in Digital Systems*. Springer Science & Business Media, 2012.

[Huf52] David A. Huffman. "A Method for the Construction of Minimum-Redundancy Codes". In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101.

[Hun05] Robert William Gainer Hunt. *The Reproduction of Colour*. John Wiley & Sons, 2005.

[HW22] Ping A. Hsieh and Ja-Ling Wu. "A Review of the Asymmetric Numeral System and Its Applications to Digital Images". In: *Entropy* 24.3 (2022), p. 375.

[HZ10] Alain Hore and Djemel Ziou. "Image quality metrics: PSNR vs. SSIM". In: *2010 20th international conference on pattern recognition*. IEEE. 2010, pp. 2366–2369.

[Jon+19] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. "Cloud Programming Simplified: A Berkeley View on Serverless Computing". In: *arXiv preprint arXiv:1902.03383* (2019).

[Kav14] Michael Kavis. "Cloud Service Models". In: *Architecting The Cloud*. John Wiley & Sons, Ltd, 2014. Chap. 2, pp. 13–22.

[Kin23] M. Scott Kingsley. *Cloud Technologies and Services: Theoretical Concepts and Practical Applications*. Springer Nature, 2023.

[LH88] Margaret Livingstone and David Hubel. "Segregation of Form, Color, Movement, and Depth: Anatomy, Physiology, and Perception". In: *Science* 240.4853 (1988), pp. 740–749.

[Lib18] Alex Libby. *Beginning SVG: A Practical Introduction to SVG using Real-World Examples*. Apress, 2018.

[Llo+18] Wes Lloyd, Shruti Ramesh, Swetha Chinthalapati, Lan Ly, and Shrideep Pallickara. "Serverless Computing: An Investigation of Factors Influencing Microservice Performance". In: *2018 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE. 2018, pp. 159–169.

[LN12] Michael F. Land and Dan-Eric Nilsson. *Animal Eyes*. Oxford University Press, 2012.

[Mag03] Pierre Magnan. "Detection of visible photons in CCD and CMOS: A comparative view". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 504.1 (2003), pp. 199–212.

[Mar15] Guillaume C. Marty. *Optimising SVG images*. 2015.

[MG11] Peter Mell and Tim Grance. *The NIST Definition of Cloud Computing*. 2011.

[Mor99] Michael E. Mortenson. *Mathematics for Computer Graphics Applications*. Industrial Press Inc., 1999.

[ORe03] Tim O'Reilly. *The Open Source Paradigm Shift*. 2003.

[ORe18] Gerard O'Regan. *The Innovation in Computing Companion*. Springer, 2018.

[Ous98] John K. Ousterhout. “Scripting: Higher-Level Programming for the 21st Century”. In: *Computer* 31.3 (1998), pp. 23–30.

[Ouw06] J.D. van Ouwerkerk. “Image super-resolution survey”. In: *Image and Vision Computing* 24.10 (2006), pp. 1039–1052.

[Pas76] Richard C. Pasco. *Source Coding Algorithms for Fast Data Compression*. Stanford University, 1976.

[PD84] Thomas Porter and Tom Duff. “Compositing Digital Images”. In: *Proceedings of the 11th annual conference on Computer graphics and interactive techniques*. 1984, pp. 253–259.

[Pet11] Dana Petcu. “Portability and Interoperability between Clouds: Challenges and Case Study”. In: *Towards a Service-Based Internet: 4th European Conference*. Springer. 2011, pp. 62–74.

[PS17] Hussachai Puripunpinyo and Mansur H. Samadzadeh. “Effect of Optimizing Java Deployment Artifacts on AWS Lambda”. In: *2017 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE. 2017, pp. 438–443.

[Ric17] Felix Richter. *Smartphones Cause Photography Boom*. 2017.

[Ris78] Jorma Rissanen. “Modeling By Shortest Data Description”. In: *Automatica* 14.5 (1978), pp. 465–471.

[Sal98] David Salomon. *Data Compression: The Complete Reference*. Springer, 1998.

[SBW19] Mohammad Shahrad, Jonathan Balkind, and David Wentzlaff. “Architectural Implications of Function-as-a-Service Computing”. In: *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 2019, pp. 1063–1075.

[Sch+23] Trever Schirmer, Nils Japke, Sofia Greten, Tobias Pfandzelter, and David Bermbach. “The Night Shift: Understanding Performance Variability of Cloud Serverless Platforms”. In: *Proceedings of the 1st Workshop on Serverless Systems, Applications and Methodologies*. 2023, pp. 27–33.

[Sha48] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423.

[Sta23] International Organization for Standardization. “Information technology – Cloud computing – Part 1: Vocabulary”. Standard. 2023.

[The95] Albert J.P. Theuwissen. *Solid-State Imaging with Charge-Coupled Devices*. Springer, 1995.

[UAG21] Dmitrii Ustiugov, Theodor Amariucai, and Boris Grot. “Analyzing Tail Latency in Serverless Clouds with STeLLAR”. In: *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE. 2021, pp. 51–62.

[Wac93] Edward Wachtel. “The First Picture Show: Cinematic Aspects of Cave Art”. In: *Leonardo* 26.2 (1993), pp. 135–140.

[Wan+18] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. “Peeking Behind the Curtains of Serverless Platforms”. In: *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 2018, pp. 133–146.

[WD05] Ron White and Timothy E. Downs. *How Digital Photography Works*. Que Corp., 2005.

[Wel84] Terry A. Welch. “A Technique for High-Performance Data Compression”. In: *Computer* 17.06 (1984), pp. 8–19.

[WJ22] Qinzhe Wu and Lizy K. John. “Performance of Java in Function-as-a-Service Computing”. In: *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*. IEEE. 2022, pp. 261–266.

[WM23] Yao Wang and Debargha Mukherjee. “The Discrete Cosine Transform and Its Impact on Visual Compression: Fifty Years From Its Invention [Perspectives]”. In: *IEEE Signal Processing Magazine* 40.6 (2023), pp. 14–17.

[WTJ14] Stefan Walraven, Eddy Truyen, and Wouter Joosen. “Comparing PaaS offerings in light of SaaS development: A comparison of PaaS platforms based on a practical case study”. In: *Computing* 96 (2014), pp. 669–724.

[Zdn03] Editors of Zdnet. *More digital than film cameras sold*. 2003.

[ZL77] Jacob Ziv and Abraham Lempel. “A Universal Algorithm for Sequential Data Compression”. In: *IEEE Transactions on Information Theory* 23.3 (1977), pp. 337–343.

[ZL78] Jacob Ziv and Abraham Lempel. “Compression of Individual Sequences via Variable-Rate Coding”. In: *IEEE Transactions on Information Theory* 24.5 (1978), pp. 530–536.